

Tafila Technical University

Faculty of Information Technology

Data Analysis for cyber security

Instructor: Dr. Shorouq Al-Eidi

Rebort BY:

Student's names: Noor Al-Shater

Cyber Security Indexes:

Four related to cyber security Indexes presented for each country

ABOUT DATASET:

The Dataset "Cyber Security Indexes" includes four indicators which illustrate the current cyber security situation around the world. The data is provided on 193 countries and territories, grouped by five geographical regions - Africa, North America, South America, Europe and Asia-Pasific.

The Cybersecurity Exposure Index **(CEI)** defines the level of exposure to cybercrime by country from 0 to 1; the higher the score, the higher the exposure (provided by 10guard). The indicator was last updated in **2020.**

The Global Cyber Security Index **(GCI)** is a trusted reference that measures the commitment of countries to cybersecurity at a global level – to raise awareness of the importance and different dimensions of the issue (provided by the International Telecommunication Union - ITU). The indicator was last updated in **2021.**

The National Cyber Security Index **(NCSI)** measures a country's readiness to address cyber threats and manage cyber incidents. It is composed of categories, capacities, and indicators (provided by NCSI). The indicator was last updated in **January 2023**.

The Digital Development Level **(DDL)** defines the average percentage the country received from the maximum value of both indices (provided by NCSI). The indicator was last updated in January **2023**

Analysis Section:

jupyter Untitled9 Last Checkpoint: 19 minutes ago

File Edit View Run Kernel Settings Help

Code

```
[2]: import pandas as pd

[73]: pd.read_csv(r"C:\Users\HP\Desktop\Cyber_security.csv")
```

[73]:

	Country	Region	CEI	GCI	NCSI	DDL
0	Afghanistan	Asia-Pasific	1.000	5.20	11.69	19.50
1	Albania	Europe	0.566	64.32	62.34	48.74
2	Algeria	Africa	0.721	33.95	33.77	42.81
3	Andorra	Europe	NaN	26.38	NaN	NaN
4	Angola	Africa	NaN	12.99	9.09	22.69
...	...	...	...	...	...	...
187	Venezuela	Africa	0.807	27.06	28.57	43.14
188	Vietnam	Asia-Pasific	NaN	94.59	36.36	47.69
189	Yemen	Asia-Pasific	NaN	NaN	7.79	NaN
190	Zambia	Africa	0.745	68.88	55.84	29.66
191	Zimbabwe	Africa	0.724	36.49	15.58	28.97

192 rows × 6 columns

* The **Pandas** library is imported using `import pandas as pd`, which provides tools for data analysis. Then, data is loaded from a CSV file using the `pd.read_csv()` function, where the file path is specified for loading. The result is the conversion of the data into a **DataFrame** structure for easy manipulation and analysis.

```
[6]: cyber_df=pd.read_csv(r"C:\Users\HP\Desktop\Cyber_security.csv")

[8]: print(cyber_df.head())
```

```
       Country        Region    CEI    GCI   NCSI    DDL
0  Afghanistan  Asia-Pasific  1.000   5.20  11.69  19.50
1      Albania        Europe  0.566  64.32  62.34  48.74
2      Algeria        Africa  0.721  33.95  33.77  42.81
3      Andorra        Europe    NaN  26.38    NaN    NaN
4       Angola        Africa    NaN  12.99   9.09  22.69
```

* The `print(cyber_df.head())` command displays the first five rows of the DataFrame `cyber_df`, allowing a quick preview of the data.

```
shape=cyber_df.shape
```

```
print("Number of rows:",shape[0])
```

```
Number of rows: 192
```

```
print("Number of columns:",shape[1])
```

```
Number of columns: 6
```

*The **cyber_df.shape** command returns the dimensions of the DataFrame cyber_df as a tuple (number of rows, number of columns). The subsequent **print()** statements display the number of rows (shape[0]) and columns (shape[1]) in the DataFrame

```
cyber_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 192 entries, 0 to 191
Data columns (total 6 columns):
 #   Column   Non-Null Count  Dtype  
---  ------   --------------  -----  
 0   Country  192 non-null    object 
 1   Region   192 non-null    object 
 2   CEI      108 non-null    float64
 3   GCI      190 non-null    float64
 4   NCSI     167 non-null    float64
 5   DDL      152 non-null    float64
dtypes: float64(4), object(2)
memory usage: 9.1+ KB
```

The **cyber_df.info()** method is used to get a comprehensive summary of the DataFrame.

- It displays the number of rows and columns in the dataset.
- It shows the column names, data types (int, float, object), and the count of non-null values in each column.
- This helps in understanding data quality and identifying columns with missing or inconsistent values.

```python
import pandas as pd

# read data
df =pd.read_csv(r"C:\Users\HP\Desktop\Cyber_security.csv")

# veryfy from missing value for each column
missing_values = df.isnull().any()

print("are there any missing value?")
print(missing_values)
```

```
are there any missing value?
Country    False
Region     False
CEI         True
GCI         True
NCSI        True
DDL         True
dtype: bool
```

This code checks for missing values in each column of the dataset.

- The data is read using **`pd.read_csv()`** and stored in a DataFrame named `df`.
- The **`df.isnull().any()`** function checks whether any missing values exist in each column, returning a list of Boolean values (True/False) for each column.
- The output shows that the columns **CEI**, **GCI**, **NCSI**, and **DDL** have missing values (True), while the columns **Country** and **Region** do not have missing values (False).

```python
import pandas as pd

# Read the file
df = pd.read_csv(r"C:\Users\HP\Desktop\Cyber_security.csv")

# Display the number of missing values in each column before deletion
print("Number of missing values in each column before deletion:")
print(df.isnull().sum())

# Remove rows with missing values
df_cleaned = df.dropna()

# Display the number of missing values after deletion
print("\nNumber of missing values in each column after deletion:")
print(df_cleaned.isnull().sum())

# Display the shape of the data before and after deletion
print("\nShape before deletion:", df.shape)
print("Shape after deletion:", df_cleaned.shape)

# Save the cleaned data if necessary
df_cleaned.to_csv(r"C:\Users\HP\Desktop\Cyber_security.csv", index=False)
```

```
Number of missing values in each column before deletion:
Country     0
Region      0
CEI        84
GCI         2
NCSI       25
DDL        40
dtype: int64

Number of missing values in each column after deletion:
Country    0
Region     0
CEI        0
GCI        0
NCSI       0
DDL        0
dtype: int64

Shape before deletion: (192, 6)
Shape after deletion: (104, 6)
```

Before deletion, the DataFrame has 192 rows and 6 columns. The Country and Region columns have no missing values, while the other columns like CEI and NCSI have missing values (84 and 25 respectively).

After deleting the rows with missing values using dropna(), all the columns are now free of missing values.

The number of rows has reduced from 192 to 104, which means that 88 rows with missing values have been deleted.

The data is now clean and free of any missing values

```python
duplicate_rows=cyber_df[cyber_df.duplicated()]
print("duplicate_rows in the dataframe:")
print(duplicate_rows)
```

```
duplicate_rows in the dataframe:
Empty DataFrame
Columns: [Country, Region, CEI, GCI, NCSI, DDL]
Index: []
```

This code identifies duplicate rows in the **`cyber_df`** DataFrame using **`cyber_df.duplicated()`,** which returns rows that are duplicated based on all columns.

The duplicate rows are stored in a new DataFrame called **`duplicate_rows`**.

The output shows that the DataFrame contains no duplicate rows, as the result is an empty DataFrame.

```
correlation_matrix = cyber_df.select_dtypes(include='number').corr()

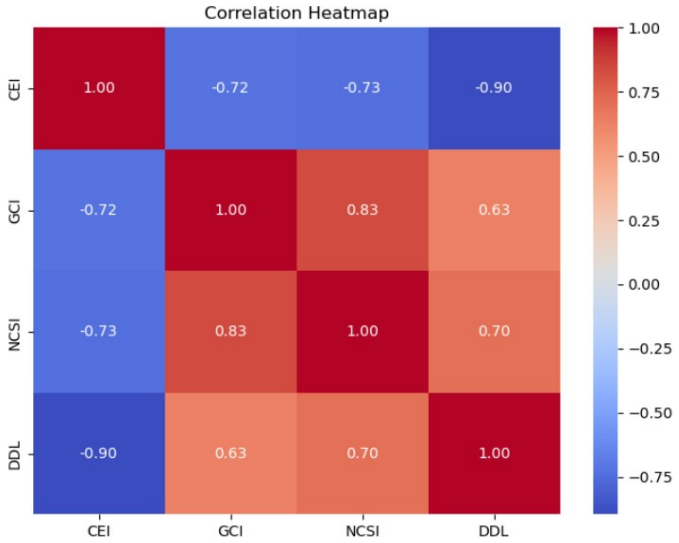
print("Correlation Matrix:\n", correlation_matrix)
```

```
Correlation Matrix:
            CEI       GCI      NCSI       DDL
CEI    1.000000 -0.723111 -0.733569 -0.896193
GCI   -0.723111  1.000000  0.830815  0.631292
NCSI  -0.733569  0.830815  1.000000  0.700703
DDL   -0.896193  0.631292  0.700703  1.000000
```

The correlation matrix shows the pairwise correlation between the numerical columns: **CEI**, **GCI**, **NCSI**, and **DDL**.

- Strong negative correlations are observed between **CEI** and **NCSI** (-0.73) and **CEI** and **DDL** (-0.90), while **GCI** and **NCSI** show a strong positive correlation (0.83).

```
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Heatmap')
plt.show()
```



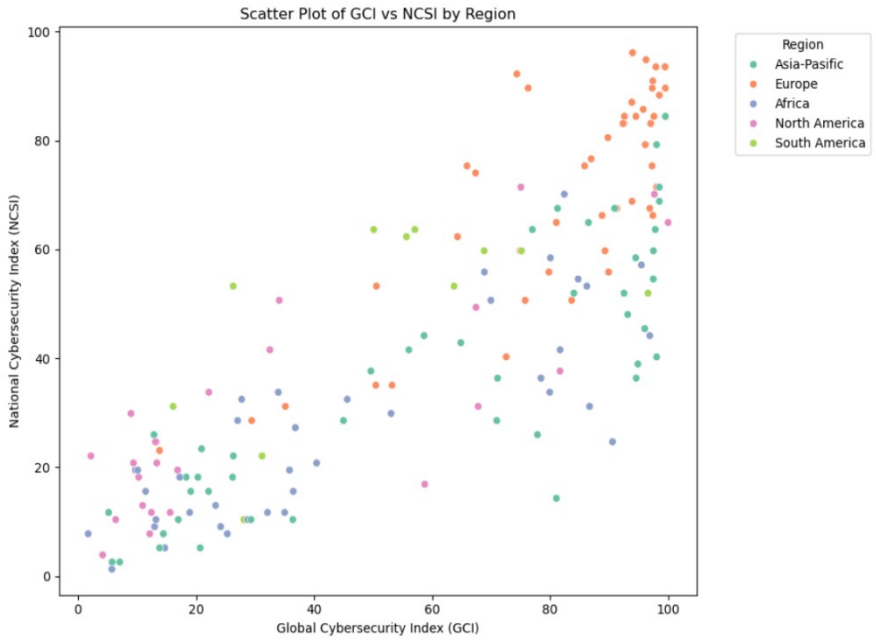
This code generates a heatmap to visualize the correlation matrix using **seaborn** and **matplotlib**.

- `sns.heatmap()` creates the heatmap with annotations (`annot=True`) to display the correlation values, and the `cmap='coolwarm'` sets the color palette. The `plt.title()` adds a title to the plot, and `plt.show()` displays the heatmap.

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# Read the data
data = pd.read_csv(r"C:\Users\HP\Desktop\Cyber_security.csv")

# Create the plot
plt.figure(figsize=(10, 7))
sns.scatterplot(x='GCI', y='NCSI', data=data, hue='Region', palette='Set2')
plt.xlabel('Global Cybersecurity Index (GCI)')
plt.ylabel('National Cybersecurity Index (NCSI)')
plt.title('Scatter Plot of GCI vs NCSI by Region')
plt.legend(title='Region', bbox_to_anchor=(1.05, 1), loc='upper left')
plt.tight_layout()
plt.show()
```



This code creates a scatter plot to visualize the relationship between **GCI** (Global Cybersecurity Index) and **NCSI** (National Cybersecurity Index), with different colors representing regions using the `hue='Region'` parameter.

- `sns.scatterplot()` plots the data, `plt.xlabel()` and `plt.ylabel()` label the axes, and `plt.title()` adds a title. The legend is placed outside the plot with `plt.legend()`, and `plt.tight_layout()` ensures the layout fits well.

```python
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

# Load the data
cyber_df = pd.read_csv(r"C:\Users\HP\Desktop\Cyber_security.csv")

# Read the data
df = pd.read_csv(r"C:\Users\HP\Desktop\Cyber_security.csv")

# Plot Pair Plot
sns.pairplot(df, diag_kind='kde', corner=True)

# Show the plot
plt.show()

# Plot Pair Plot with hue by Region
sns.pairplot(cyber_df, hue='Region', diag_kind='kde')

# Set the title for the plot
plt.suptitle('Pairplot of Cyber Security Dataset', y=1.02)  # y to adjust the title's position
plt.show()
```



This code generates plots for the **Cyber Security** dataset using **seaborn's** `pairplot()` function.

- plot adds color grouping by the **Region** column using `hue='Region'` and also includes KDE on the diagonals.

```python
# Load the data
cyber_df = pd.read_csv(r"C:\Users\HP\Desktop\Cyber_security.csv")

# Remove spaces from column names
cyber_df.columns = cyber_df.columns.str.strip()

# Select the column to analyze for outliers (e.g., NCSI)
column_to_analyze = 'NCSI'

# Calculate the interquartile range (IQR)
Q1 = np.percentile(cyber_df[column_to_analyze].dropna(), 25, interpolation='midpoint')
Q3 = np.percentile(cyber_df[column_to_analyze].dropna(), 75, interpolation='midpoint')
IQR = Q3 - Q1

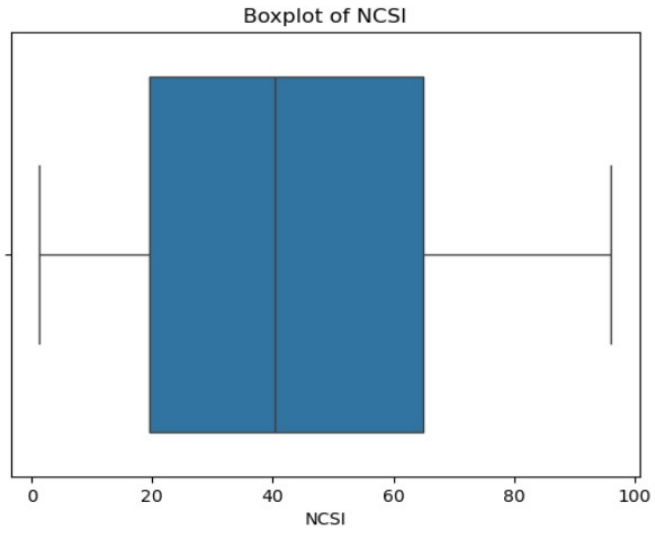
# Define the upper and lower bounds
upper = np.where(cyber_df[column_to_analyze] > (Q3 + 1.5 * IQR))
lower = np.where(cyber_df[column_to_analyze] < (Q1 - 1.5 * IQR))

# Print the old and new shape after removing outliers
print("Old Shape: ", cyber_df.shape)
cyber_df.drop(upper[0], inplace=True)
cyber_df.drop(lower[0], inplace=True)
print("New Shape: ", cyber_df.shape)

# Plot the Boxplot
sns.boxplot(x=cyber_df[column_to_analyze])

plt.title(f'Boxplot of {column_to_analyze}')
plt.show()
```

```
Old Shape:  (192, 6)
New Shape:  (192, 6)
```



The code first loads the **Cyber Security** dataset and removes any spaces from column names.

It then calculates the interquartile range (IQR) for the **NCSI** column to detect outliers, defines upper and lower bounds, and removes rows that contain outliers.

Finally, it prints the dataset shape before and after removing the outliers, and visualizes the distribution of **NCSI** using a boxplot.

```python
from scipy import stats
import pandas as pd

# Read the data
cyber_df = pd.read_csv(r"C:\Users\HP\Desktop\Cyber_security.csv")

# Select the appropriate column to analyze for outliers (e.g., NCSI)
column_to_analyze = 'NCSI'

# Handle missing values
cyber_df = cyber_df.dropna(subset=[column_to_analyze])

# Calculate Z-Scores
z_scores = stats.zscore(cyber_df[column_to_analyze])

# Define the threshold for outliers
threshold = 3

# Remove outliers using Z-Scores
df_no_outliers = cyber_df[(z_scores < threshold) & (z_scores > -threshold)]

# Print the new shape
print("Old Shape:", cyber_df.shape)
print("New Shape (Without Outliers):", df_no_outliers.shape)

# Display the data after removing outliers
print(df_no_outliers)
```

```
Old Shape: (167, 6)
New Shape (Without Outliers): (167, 6)
                Country         Region    CEI    GCI   NCSI    DDL
0           Afghanistan   Asia-Pasific  1.000   5.20  11.69  19.50
1               Albania         Europe  0.566  64.32  62.34  48.74
2               Algeria         Africa  0.721  33.95  33.77  42.81
4                Angola         Africa    NaN  12.99   9.09  22.69
5    Antigua and Barbuda  North America    NaN  15.62  11.69  57.10
..                   ...            ...    ...    ...    ...    ...
187            Venezuela         Africa  0.807  27.06  28.57  43.14
188              Vietnam   Asia-Pasific    NaN  94.59  36.36  47.69
189                Yemen   Asia-Pasific    NaN    NaN   7.79    NaN
190               Zambia         Africa  0.745  68.88  55.84  29.66
191             Zimbabwe         Africa  0.724  36.49  15.58  28.97

[167 rows x 6 columns]
```

This code calculates the **Z-Scores** for the **NCSI** column in the dataset to identify outliers. It then removes the rows **with Z-Scores** greater than the threshold (3) or less than -3, and prints the shape of the dataset before and after outlier removal, displaying the cleaned data

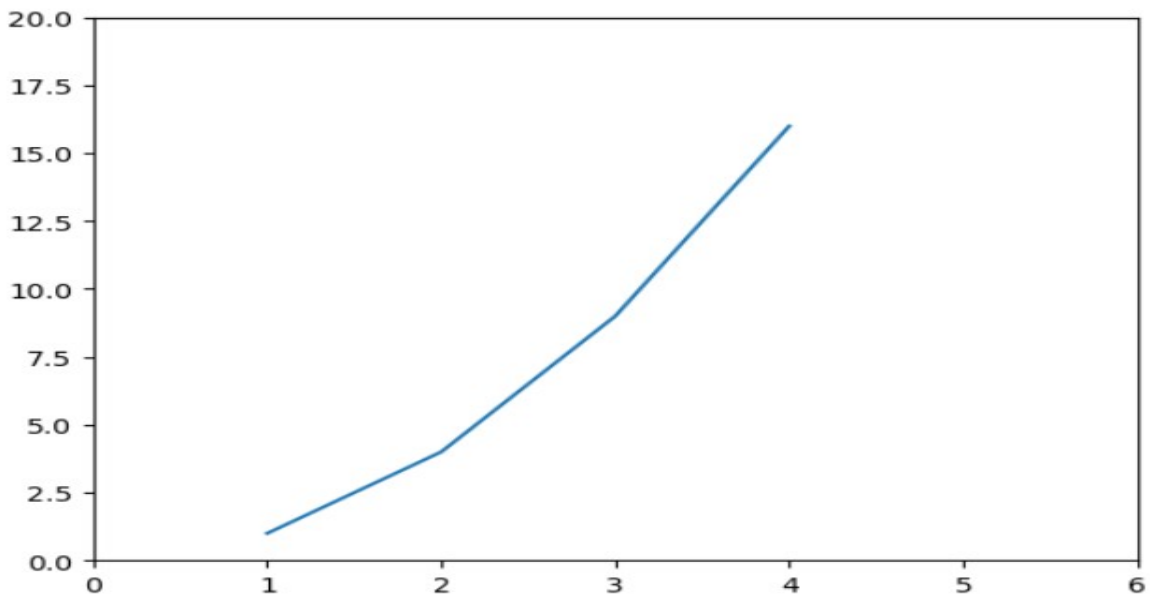
```python
# Python program to demonstrate matplotlib.pyplot module
import matplotlib.pyplot as plt

# Set up the x and y axis data
x = [1, 2, 3, 4]
y = [1, 4, 9, 16]

# Plot the graph
plt.plot(x, y)

# Set the axis limits
plt.axis([0, 6, 0, 20])

# Show the plot
plt.show()
```



This code uses **matplotlib.pyplot** to plot a line graph with x and y values.

The axis limits are set to `[0, 6]` for the x-axis and `[0, 20]` for the y-axis using **`plt.axis()`,** and the graph is displayed with **`plt.show()`.**

```python
import matplotlib.pyplot as plt
import pandas as pd

# Read the data
df = pd.read_csv(r"C:\Users\HP\Desktop\Cyber_security.csv")

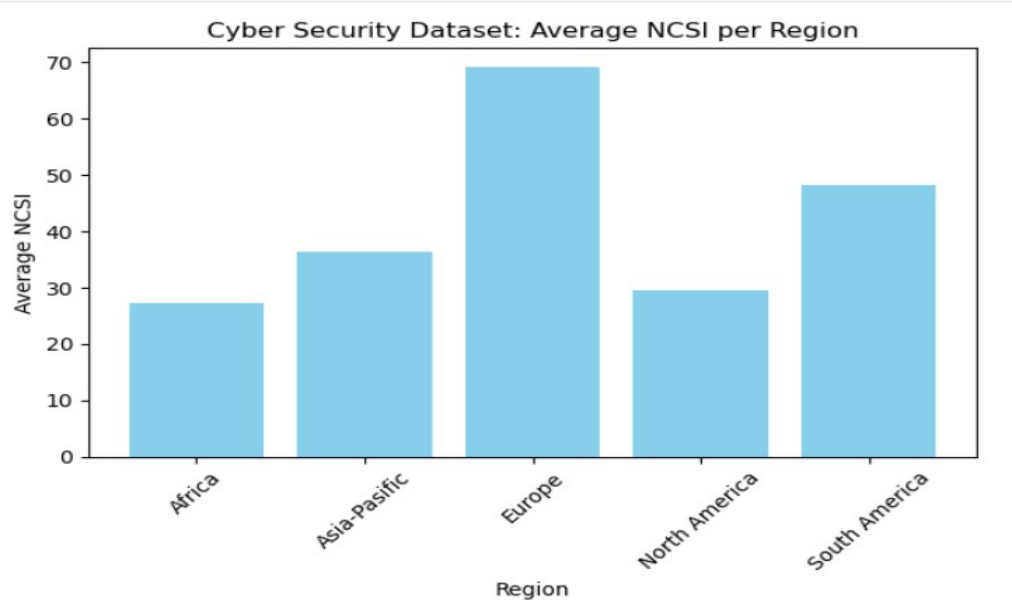
# Calculate the average NCSI for each Region
avg_ncsi = df.groupby('Region')['NCSI'].mean()

# Create a bar chart
plt.bar(avg_ncsi.index, avg_ncsi.values, color='skyblue')

# Add titles and labels
plt.title("Cyber Security Dataset: Average NCSI per Region")
plt.xlabel("Region")
plt.ylabel("Average NCSI")

# Rotate the category labels for better readability
plt.xticks(rotation=45)

# Show the plot
plt.tight_layout()
plt.show()
```



This code creates a bar chart to display the average **NCSI** (National Cyber Security Index) for each region in the dataset.

1. It calculates the average **NCSI** for each region using `groupby('Region')['NCSI'].mean()`.
2. A bar chart is plotted with regions on the x-axis and their corresponding average **NCSI** values on the y-axis.

3. Titles and axis labels are added, and the x-axis labels are rotated for better readability. Finally, the plot is displayed using `plt.show()`

```
import matplotlib.pyplot as plt
import pandas as pd

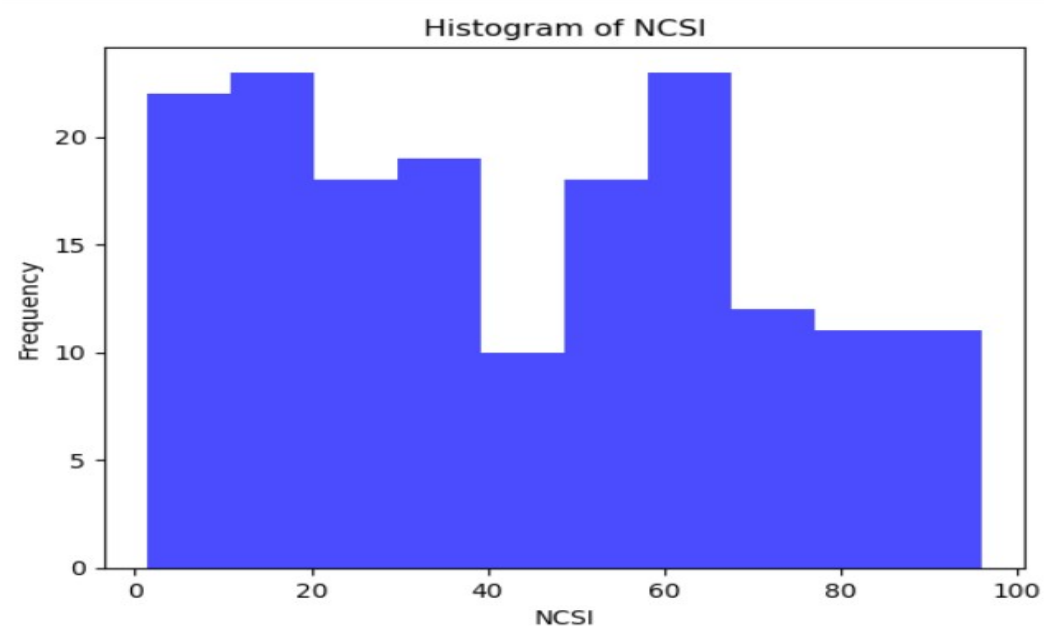
# Read the data
df = pd.read_csv(r"C:\Users\HP\Desktop\Cyber_security.csv")

# Plot the histogram
plt.hist(df["NCSI"], bins=10, color='blue', alpha=0.7)

# Add the title
plt.title("Histogram of NCSI")

# Add x and y axis labels
plt.xlabel("NCSI")
plt.ylabel("Frequency")

# Show the plot
plt.show()
```



This code creates a histogram to visualize the distribution of the **NCSI** values in the dataset.

- The `plt.hist()` function is used with 10 bins to group the data, and the color is set to blue with transparency (`alpha=0.7`).
- Titles and axis labels are added for clarity, and the plot is displayed using `plt.show()`.

```python
import pandas as pd
import matplotlib.pyplot as plt

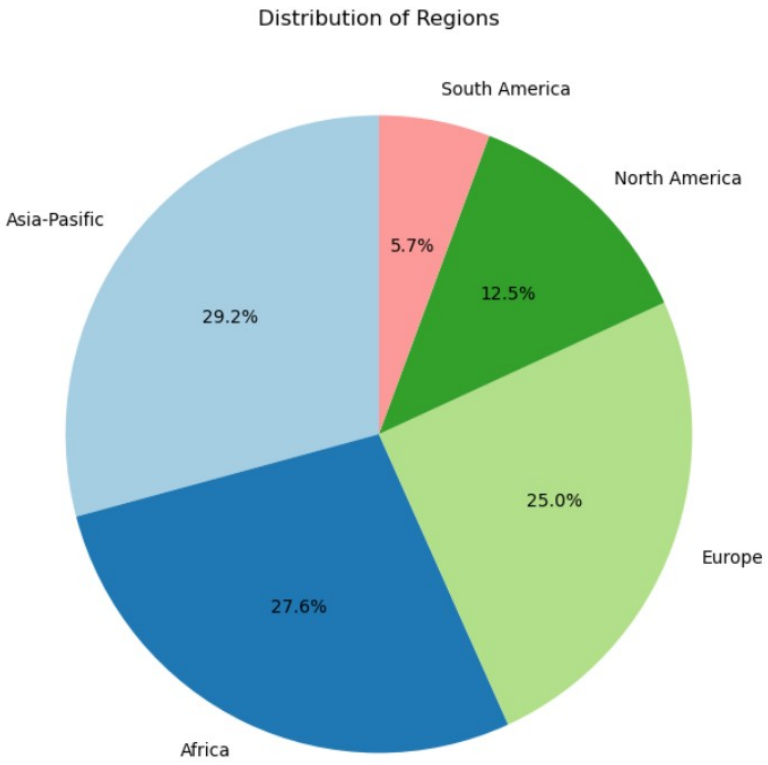
# Read the data
df = pd.read_csv(r"C:\Users\HP\Desktop\Cyber_security.csv")

# Count the values in a specific column (e.g., "Region" or "Category")
category_counts = df['Region'].value_counts()

# Plot the Pie Chart
plt.figure(figsize=(8, 8))
plt.pie(category_counts, labels=category_counts.index, autopct='%1.1f%%', startangle=90, colors=plt.cm.Paired.colors)

# Add title
plt.title("Distribution of Regions")

# Show the plot
plt.show()
```



This code generates a **pie chart** to visualize the distribution of regions in the dataset.

- It first counts the occurrences of each region using `value_counts()` on the **Region** column, then uses `plt.pie()` to plot the distribution, showing percentages (`autopct='%1.1f%%'`), and colors are chosen from the `Paired` colormap.
- A title is added, and the plot is displayed using `plt.show()`.

Conclusion

Throughout this analysis of **Cyber Security** data, we utilized various statistical and visual techniques to gain deeper insights into the dataset. We identified significant relationships between indices, examined the distribution of cybersecurity across regions, and handled outliers to ensure data accuracy. This effort helped us extract clear patterns that can be relied upon to make informed decisions aimed at enhancing cybersecurity standards globally.